# 使い方

## スイッチの切り替え：**Switchcontrol.py**

◯特性評価/実験、**TM110/TM210**を切り替える

**%** python Switchcontrol.py AMP TM110
・引数1　VNA or AMP
・引数2　TM110 or TM210

◯スイッチ用電源の**CH2・CH3・CH4**の出力を全て**OFF**にする

% python Switchcontrol.py OFF

◯**python**起動後
>>>import Switchcontrol
>>>Switchcontrol.set_switch("VNA", "TM110")
・引数1　VNA or AMP
・引数2　TM110 or TM210
または
 >>>Switchcontrol.all_off()

## VNAの設定・測定・CSV保存：test_main.py

◯**VNA**接続 → 測定条件の設定 → 全**4ch**を各**10**回測定 → **CSV**保存をまとめて行う

% python test_main.py
・保存ファイル名の末尾を指定する　% python test_main.py TM110_check
　引数：ファイル名の末尾に付ける文字列

## VNAの個別操作：vna_tools.py

①**Python**起動後、**VNA**と接続
>>> import pyvisa
>>> import vna_tools
>>> rm = pyvisa.ResourceManager('@py')
>>> znb = vna_tools.get_vna_resource(rm, 'TCPIP0::192.168.12.4::inst0::INSTR')

◯測定条件・トレース・画面・マーカーを設定する
>>> vna_tools.setup_vna(znb)
・TM110/TM210それぞれのWide/Narrowを全4chに設定する。

◯全**4ch**を各**10**回測定し、**CSV**を保存する
>>> csv_path = vna_tools.measure_and_save(znb)

◯現在の**Narrow S11**から、最小振幅の周波数を取得する
>>> f0, amp_min = vna_tools.find_min_freq(znb, 2, "Trc4")
・TM110：2, "Trc4"
・TM210：4, "Trc9"

◯操作終了後、通信を閉じる
>>> znb.close()
>>> rm.close()

## Piezo調整

①**ANC350**を初期化する：**piezo_move.py**
・ANC350をUSBでWindows PCに接続する。
・Windows PCで piezo_move.py を実行し、初期化する。
・USBをWindows PCからLinux PCへ差し替える。

②**Linux PC**で受信プログラムを起動する：**frequency_controller.py**

% python frequency_controller.py

・Waiting for frequency command...と表示されたら、別タブから指示を送る。
・このターミナルは起動したままにする。
・この受信側のターミナルに、現在の周波数・目標との差・移動step数・最終結果が表示される。
・調整終了後は、次の指示を待ち受ける。

③別タブで**Python**を起動し、送信側を準備する

◯**python**起動後
>>> from frequency_command import FrequencyCommandProducer
>>> producer = FrequencyCommandProducer()
>>> producer.send_target("TM110", 1897400)
・引数1：TM110 or TM210
・引数2：目標周波数［kHz］

◯送信側の通信を閉じる
>>> producer.close()

## パワーのfeedback

①パワー測定値の送信を開始する：**RaS_NRP2_Operator.py**

% python RaS_NRP2_Operator.py

・このターミナルは起動したままにする。

②スイッチを**SG**側・使用するモードに切り替える

% python Switchcontrol.py AMP TM110

・TM210の場合はTM210を指定する。

③別タブで**Python**を起動し、フィードバックを準備する

◯**python**起動後
>>> from power_feedback import PowerFeedbackController
>>> feedback = PowerFeedbackController()
>>> feedback.start(frequency_ghz=1.8974, target_power_w=7e-6, initial_sg_power_dbm=-25.0)

・frequency_ghz：SGの周波数［GHz］
・target_power_w：NRP2 CH2で測定する目標パワー［W］
・initial_sg_power_dbm：開始時のSG出力［dBm］
・周波数設定 → 初期SG出力設定 → SG出力ON → フィードバック開始まで行う。

◯設定の変更
変更後の設定は、次の制御計算から使われる

目標パワーを変更する
>>> feedback.set_target_power(7e-6)
・引数：目標パワー［W］

**PID**の係数を変更する
>>> feedback.set_pid(kp=1.0, ki=0.0, kd=0.0)

制御の更新間隔を変更する
>>> feedback.set_interval(1.0)
・引数：更新ごとの待ち時間［秒］

◯現在の状態を確認する
>>> feedback.get_status()
・SG出力ON/OFF、フィードバック状態、周波数、SG出力、目標値、最新の測定値などを表示する。

◯フィードバックを停止し、**SG**出力を**OFF**にする
>>> feedback.stop()

◯ログのパスを確認
>>> print(feedback.log_path)

◯測定ログのリアルタイム確認
別のタブで
% tail -f ログのパス